

Práctica 1

Seguimiento de un vehículo en la ciudad

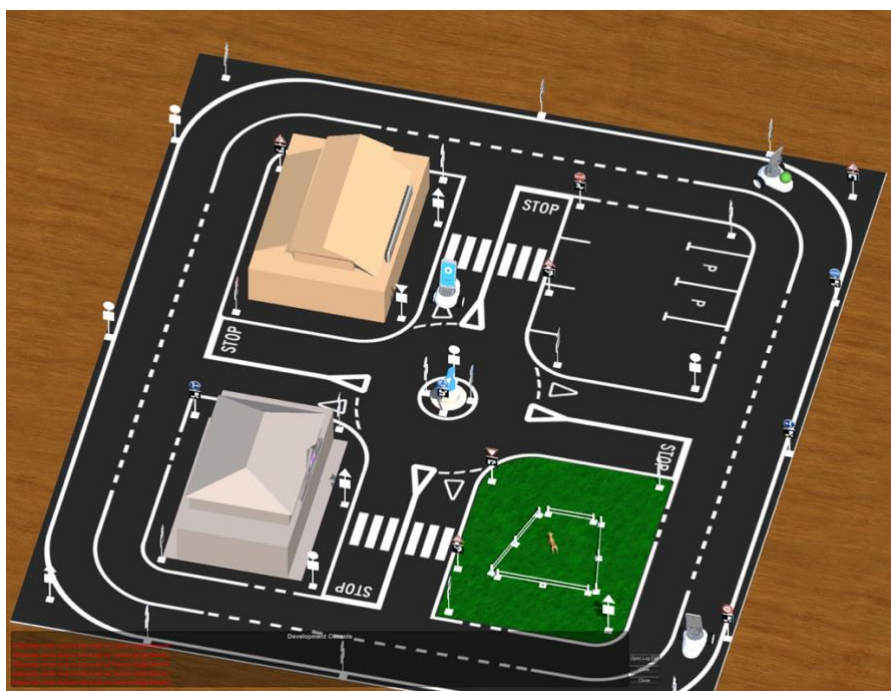
Aprendizaje por refuerzo en espacios continuos. Desarrollo en simulación y transferencia al entorno real

Entrega	Viernes 16 de octubre de 2026 a las 23:55, por el campus online
Formato	Archivo .zip con (1) memoria, (2) declaración de uso de IA, (3) código fuente comentado y (4) evidencias: modelos, registros y vídeos
Memoria	Máximo 6 páginas, más los anexos que hagan falta, en Calibri 11 pt con interlineado sencillo. Declaración de uso de IA: documento breve (máximo una página) que indique qué asistentes se han usado, para qué partes concretas de la práctica (código, depuración, redacción de la memoria, diseño de la recompensa...), qué se ha aceptado, qué se ha modificado o descartado y por qué, y qué errores de los asistentes se han detectado y cómo; si no se ha usado ninguno, debe declararse expresamente.
Trabajo	Por parejas

1. Objetivo

En la ciudad de RoboSim hay un RoboSim, el **vehículo guía**, que circula continuamente por el anillo exterior con una pelota verde en su parte trasera. La tarea consiste en que otro RoboSim, el **seguidor**, aprenda a circular detrás de él: **manteniendo la separación dentro de la banda objetivo, sin chocar, sin perderlo de vista y sin salirse de su carril**.

El seguidor se controla con una política obtenida mediante aprendizaje por refuerzo en un espacio de acciones continuo. El desarrollo se realiza en simulación, pero la práctica no termina ahí: la política debe **ejecutarse después en el entorno real**, es decir, sobre la ciudad construida en el laboratorio, sin volver a entrenar.



2. El escenario

- **El vehículo guía** circula solo por un carril del anillo exterior y no reacciona al seguidor: si éste se detiene, el guía sigue su camino. Su conducción se configura en el simulador con un modo y unos límites de velocidad, entre 10 y 40 en las unidades del simulador.
- **Modos de conducción:** en **crucero** aplica la velocidad máxima en las rectas y la mínima en las curvas; en **variable** fluctúa entre ambas, frenando también en las curvas. Existe un tercer modo que añade paradas aleatorias, que **no se utiliza en esta práctica**: el guía nunca se detiene.
- **La pelota verde** es lo que permite reconocerlo. La detecta el módulo de blobs de color del propio robot. Puede valorarse además el uso de los infrarrojos para detectar al guía cuando está muy cerca.
- **El seguidor** es el robot al que se conecta el nodo puente de robobo_ros2. Todo el control y toda la percepción pasan por ROS 2.
- **En el entorno real** el vehículo guía es otro Robobo con la misma pelota, conducido por el profesorado o por un programa que recorre el anillo.

3. Herramientas

- **Gymnasium:** estructura del entorno de aprendizaje.
- **Stable-Baselines3:** implementación de los algoritmos de aprendizaje por refuerzo.
- **robobo_ros2:** nodo puente entre ROS 2 y el robot, simulado o real. Es la única vía de acceso a los sensores y actuadores del seguidor. Su interfaz está resumida en el anexo A.
- **El módulo sim del nodo puente:** da acceso por ROS 2 a la información del simulador, la posición y la orientación de un robot, y permite colocarlo y reiniciar la escena. No hace falta instalar ninguna biblioteca aparte. No existe en el entorno real.

No se proporciona código de partida. Todo el software de la práctica lo desarrolla cada grupo: el entorno de Gymnasium, la comunicación con ROS 2, las herramientas de medida del sensor, la colocación del seguidor en la escena y el evaluador del anexo B. El uso de asistentes de IA para construirlo está permitido en los términos del apartado 10.

4. Qué puede entrar en la observación

El espacio de observación sólo puede contener información de los sensores del seguidor, recibida a través de ROS 2. La posición del guía o del seguidor, la geometría del carril, la separación real o el error lateral son información privilegiada del simulador y no pueden formar parte de la observación.

La información privilegiada que ofrece el módulo sim, y cualquier magnitud que se derive de ella, sí puede emplearse para preparar el inicio de cada episodio, para calcular la recompensa, para decidir cuándo termina un episodio y para registrar métricas. El criterio es sencillo: **lo que la política necesita para decidir debe existir también en el robot real**; lo que sólo sirve para entrenar o para medir, no.

Incumplir esta restricción hace imposible la transferencia al entorno real e invalida el apartado 5.5.

5. Desarrollo

5.1. Caracterización del sensor

Antes de diseñar nada hay que saber qué mide el sensor. Se recomienda construir una herramienta de medida propia que permita determinar en qué rango de separaciones se detecta la pelota, cómo varían su posición y su tamaño en la imagen con la distancia, cuánto ruido tienen, en qué se diferencian las rectas de las curvas y cómo influye la inclinación de la cámara. Los resultados deben aparecer en la memoria y justificar las decisiones del apartado siguiente.

5.2. Entorno de Gymnasium

Construir un entorno de Gymnasium que envuelva al seguidor. Hay que definir y justificar por escrito:

- **Espacio de observación:** qué información recibe la política, de qué sensores procede, cómo se normaliza y qué ocurre cuando la pelota no se ve.
- **Espacio de acción:** obligatoriamente continuo, con sus límites físicos justificados.
- **Función de recompensa:** con cada término separado y el valor numérico de cada coeficiente. Debe recoger los cuatro objetivos del apartado 1.
- **Condiciones de terminación y truncamiento** de los episodios.
- **Reinicio de los episodios:** cómo queda el escenario al empezar cada episodio durante el entrenamiento, y qué se hace mientras el guía sigue circulando.

Hay un requisito técnico que se comprobará expresamente: **la observación que devuelve step() debe corresponder a un instante posterior a la aplicación de la acción**. ROS 2 entrega los mensajes cuando el ejecutor los procesa, no cuando Gymnasium los pide, y devolver una lectura anterior a la acción es un error que no produce ningún mensaje: el entrenamiento se ejecuta con normalidad y no aprende. La memoria debe explicar cómo se garantiza.

El entorno debe además poder ejecutarse en las condiciones del protocolo de evaluación del anexo B, y poder ejecutarse sin simulador, con el reinicio a cargo de una persona, para la prueba en el entorno real.

5.3. Entrenamiento

- **Algoritmo:** libre, entre los de Stable-Baselines3 que admiten acciones continuas y se hayan visto en clase. Hay que justificar la elección teniendo en cuenta que cada paso del entorno cuesta tiempo real.
- **Configuración del guía:** durante el entrenamiento se puede usar cualquier modo y cualquier par de velocidades dentro del rango, y conviene experimentar con ello. La configuración empleada se declara en la memoria. La evaluación usa siempre la del anexo B.
- **Coste:** medir los segundos por paso antes de fijar el presupuesto de entrenamiento, y declararlo.
- **Línea base:** evaluar una política aleatoria con el protocolo del anexo B. Un entrenamiento que no la supere claramente no ha aprendido.
- **Diagnóstico:** si algo no funciona, la memoria debe explicar por qué, con evidencia. Un resultado negativo bien analizado es un resultado válido.

5.4. Evaluación con el protocolo común

La política final se evalúa siguiendo el **protocolo del anexo B**: ocho episodios de 60 s con cuatro configuraciones del vehículo guía fijadas, iguales para todos los grupos, medidos con la verdad del terreno y con criterios de fallo propios del protocolo, con independencia de la recompensa de cada grupo. El evaluador lo implementa cada grupo, pero el protocolo es común y no admite variaciones: las métricas deben calcularse exactamente

como se definen, para que los resultados sean comparables. Los resultados de la línea base y de la política final deben presentarse juntos.

5.5. Transferencia al entorno real

Ejecutar la política en el entorno real **sin volver a entrenar**. Antes hay que repetir la caracterización del sensor sobre el robot real, porque los valores no coinciden con los del simulador. Conviene que todo lo que dependa de la calibración esté en un archivo de configuración y no en el código.

Hay que documentar qué se degrada y por qué. **Se evalúa el análisis, no el éxito**: una transferencia parcial bien explicada puntúa más que una que funciona sin explicación. Se valorarán las medidas tomadas durante el entrenamiento para hacer posible la transferencia.

6. Aspectos que hay que afrontar

Los siguientes fenómenos aparecerán durante el desarrollo. Cómo tratarlos forma parte del trabajo, y la memoria debe mostrar cómo se han identificado y abordado.

- **El guía no espera.** Una política a medio entrenar lo pierde con frecuencia, y una vez perdido no reaparece. Además, su velocidad no es constante: frena en las curvas y acelera en las rectas, de modo que la separación no se mantiene sola.
- **Las curvas.** Seguir al vehículo guía no es lo mismo que seguir el camino por el que ha pasado. El error lateral en las curvas debe medirse y analizarse.
- **Frecuencias y retardos.** Las frecuencias del anexo A son las de publicación del nodo puente y están medidas en tiempo real, no en tiempo simulado: no cambian al acelerar el simulador, pero a mayor velocidad el robot recorre más distancia entre dos lecturas consecutivas. El periodo de control es una decisión de diseño y hay que declararlo. El temporizador de seguridad de `cmd_vel`, de 0,5 s, también cuenta tiempo real.
- **Un blob por color.** El detector entrega un único blob verde por imagen. En la ciudad hay otros elementos verdes.
- **La velocidad del simulador.** A velocidad x4 cada paso mueve al robot cuatro veces más lejos y se percibe con cuatro veces menos muestras por metro recorrido: cambia la dinámica del problema, no sólo el tiempo de entrenamiento. Es una de las diferencias entre la simulación y el entorno real que hay que tener en cuenta.

7. Condiciones

- El espacio de acción debe ser continuo.
- El nodo puente se lanza sólo con los módulos necesarios. El de cámara no hace falta para detectar blobs.
- La interfaz de actuación es libre: el tópico `cmd_vel`, los servicios de movimiento de ruedas o las acciones equivalentes. Hay que declarar cuál se usa y por qué.
- El multiplicador de velocidad del simulador se declara en la memoria y es el mismo en entrenamiento y en evaluación.
- Los parámetros que dependen de la calibración se guardan en un archivo de configuración.
- El protocolo de evaluación del anexo B se sigue sin modificaciones.

8. Experimentos adicionales

Además de mostrar la solución final en el robot real, hay que incluir una tabla con al menos **tres variantes** del diseño evaluadas con el protocolo del anexo B, que justifiquen experimentalmente decisiones que de otro modo quedarían en afirmaciones. Por ejemplo: dos definiciones distintas del espacio de observación, dos formulaciones de la recompensa, dos algoritmos con el mismo presupuesto de pasos, o distintas condiciones iniciales durante el entrenamiento. Cada fila indica qué cambia respecto a la configuración de referencia y qué efecto tiene.

9. Representación de los resultados

Todos los resultados van en la memoria, comentados: las curvas de aprendizaje de los entrenamientos y las tablas de métricas del anexo B, para la línea base, la solución final y las variantes del apartado 8, desglosadas por configuración del guía. Lo que se ve mejor en movimiento, como las trayectorias, va en los vídeos.

9.1. Curvas de aprendizaje

Una curva de aprendizaje muestra cómo evoluciona el rendimiento de la política a lo largo del entrenamiento. Sirve para comprobar que el agente aprende, para detectar cuándo deja de mejorar o se vuelve inestable, y para comparar configuraciones. Cada entrenamiento de la solución final y de las variantes del apartado 8 se acompaña de curvas que cumplan los requisitos siguientes.

- **Magnitudes:** como mínimo, la recompensa acumulada por episodio y la duración del episodio, que son las que registra Stable-Baselines3 y muestra TensorBoard (rollout/ep_rew_mean y rollout/ep_len_mean). Se recomienda añadir la fracción de episodios que terminan por cada causa (colisión, salida del carril, pérdida del guía, límite de tiempo), porque una recompensa media que sube puede ocultar un cambio en el modo en que terminan los episodios.
- **Eje horizontal:** pasos del entorno, es decir, llamadas a step(). No se admiten episodios ni tiempo de reloj, porque la duración de los episodios cambia durante el entrenamiento y el tiempo depende del ordenador. Las curvas que se comparan en una misma figura comparten la escala de este eje.
- **Eje vertical:** nombre de la magnitud y su unidad. La recompensa es adimensional; la duración se expresa en pasos o en segundos, indicando la duración de un paso; las fracciones, en tanto por uno o en porcentaje.
- **Suavizado:** los valores por episodio tienen mucho ruido. Se representa una media móvil, indicando el tamaño de la ventana en episodios, y detrás, con un trazo tenue, la serie sin suavizar.
- **Semillas:** la configuración final se entrena con al menos tres semillas distintas. La figura muestra la media y una banda de dispersión, indicando si corresponde al mínimo y el máximo o a la desviación típica. Para las variantes del apartado 8 basta una semilla, y esta limitación se tiene en cuenta al interpretar las diferencias.
- **Referencia:** una línea horizontal con la recompensa media por episodio que obtiene la política aleatoria en el propio entorno de entrenamiento.
- **Origen de los datos:** los valores se registran durante el entrenamiento, por ejemplo, con el envoltorio Monitor de Stable-Baselines3, que escribe un archivo CSV con una fila por episodio, o exportando el registro de TensorBoard. Las figuras de la memoria se generan con código propio a partir de esos datos, con una biblioteca de gráficos como Matplotlib o Seaborn. TensorBoard es una herramienta útil para seguir el entrenamiento mientras se ejecuta, pero sus capturas de pantalla no se admiten como figuras, porque no permiten combinar varias semillas en una banda, superponer la referencia ni fijar rótulos y unidades.
- **Presentación:** cada figura va numerada y con un pie que indica qué configuración se muestra, cuántas semillas, el tamaño de la ventana y el número total de pasos. Cada figura se comenta en el texto: a partir

de qué momento la política supera a la referencia, si el rendimiento se estabiliza y si hay caídas, y a qué se atribuyen.

Una curva de aprendizaje no sustituye a la evaluación. Durante el entrenamiento, la política actúa con exploración y se mide con la recompensa de cada grupo; el protocolo del anexo B usa la política determinista y cuatro métricas comunes a todos los grupos, que son las que miden la tarea. Una recompensa alta con episodios sin completar delata una recompensa mal ajustada.

9.2. Evidencias

Las figuras y las tablas de la memoria deben poder rehacerse a partir de lo que se entrega. Por eso se guardan y se entregan los registros que las generan: el registro por episodio de cada entrenamiento y el registro por paso de cada evaluación, en CSV o en el formato que produzca vuestro código, junto con los modelos y su configuración. No hay un formato obligatorio; lo que se exige es que estén y que el código que los lee venga con ellos.

10. Uso de asistentes de programación

El uso de asistentes basados en modelos de lenguaje está permitido, también para construir todo el software de la práctica, y debe declararse en un documento específico, descrito en el apartado 11.

Un asistente produce con rapidez un entorno de Gymnasium de aspecto correcto, y con la misma rapidez introduce errores que no generan ningún mensaje y que sólo se detectan porque la política no aprende. La evaluación pondera el diseño, la evidencia experimental y el diagnóstico, no la redacción del código. Los resultados de la evaluación y la tabla de experimentos exigen ejecuciones reales.

11. Entrega

Un archivo .zip con cuatro cosas:

1. **Memoria** en PDF, de seis páginas como máximo más los anexos que hagan falta, en Calibri de 11 pt con interlineado sencillo: texto justificado, figuras y tablas numeradas con pie, valores numéricos explícitos en la función de recompensa y todos los resultados del apartado 9 comentados.
2. **Declaración de uso de IA:** documento aparte, de una página como máximo, con lo que indica el apartado 10.
3. **Código fuente** comentado, con instrucciones para reproducir el entrenamiento y la evaluación, incluido el evaluador del anexo B.
4. **Evidencias:** los modelos entrenados con su configuración y sus estadísticas de normalización, los registros de entrenamiento y de evaluación que sostienen las figuras y las tablas, y dos vídeos, uno en simulación y otro en el entorno real.
5. **Prueba real:** cada grupo deberá mostrar el funcionamiento de la policy obtenida en el robot real a los profesores de la asignatura. Esa demostración se ejecutará al menos 5 veces para mostrar la robustez de la solución obtenida.

Anexo A. Interfaz de ROS 2

Con los valores por defecto del nodo puente, el espacio de nombres es /robobo/robot_0: los recursos del robot cuelgan de base/ y smartphone/, y los del simulador de sim/. Los relevantes para esta práctica son:

Recurso	Tipo	Contenido
smartphone/color_blobs	BlobArray, 5 Hz	Lista de blobs: color, x, y, size. Se publica también vacía
smartphone/set_active_color_blobs	servicio	Colores a detectar: red, green, blue, custom (bool)
base/cmd_vel	Twist	linear.x en m/s y angular.z en rad/s. Se detiene si no recibe órdenes en 0,5 s
base/move_wheels	servicio	Velocidad de cada rueda, en unidades del robot
base/move_wheels_time	servicio	Lo mismo, durante un tiempo dado. También como acción
base/move_tilt	servicio	angle y speed de tipo float32
base/move_pan	servicio	angle y speed de tipo int32
base/wheel/{left,right}/speed	Int32	Velocidad medida de cada rueda

Para actuar se puede usar cmd_vel, que trabaja en m/s y rad/s y detiene al robot si deja de recibir órdenes, o los servicios de ruedas, que trabajan directamente en velocidades de rueda. La elección es libre; lo que cambia es la cinemática que hay que resolver y el comportamiento cuando no llega ninguna orden.

```
ros2 run robobo_ros2 robobo_container --ros-args \
  -p ip:=host.docker.internal -p modules:="['blob','sim']"
```

El módulo sim da acceso al simulador sin instalar nada más. Trabaja en milímetros y grados, con el suelo en el plano (x, z) y la orientación en la componente y de la rotación:

Recurso	Tipo	Contenido
sim/robot_location	RobotLocation	id, position y rotation del robot de ese contenedor
sim/pose	Pose	Lo mismo, con la orientación en cuaternión
sim/set_robot_location	servicio	Coloca ese robot: position y rotation
sim/reset_simulation	servicio	Reinicia la escena

Cada contenedor publica la posición del robot al que está conectado, el que indican sus parámetros robot_name y robot_id. Para conocer también la del guía hace falta lanzar un segundo contenedor con el índice del otro robot y sólo el módulo sim.

Anexo B. Protocolo de evaluación

B.1. Magnitudes de referencia

Magnitud	Definición
Línea central del carril	La trayectoria cerrada que describe el vehículo guía en una vuelta completa
Separación	Distancia entre las proyecciones de ambos robots sobre la línea central; negativa si el seguidor va delante
Banda objetivo	Separación entre 0,25 m y 0,50 m
En el carril	La distancia del centro del seguidor a la línea central no supera (ancho del carril – ancho del robot) / 2. En la calzada: el centro está dentro de los dos carriles del anillo
Parámetros	Carril 0,22 m · robot 0,16 m · colisión a 0,20 m entre centros

B.2. Condiciones de la evaluación

Cada episodio empieza con un reinicio de la simulación: ambos robots parten de donde los coloca el escenario y el seguidor parte parado. Lo único que se fija es la configuración del guía, que nunca incluye paradas. La política se ejecuta en modo determinista y sin aleatorización de la percepción. Se evalúan cuatro configuraciones, con dos episodios de 60 s cada una: ocho episodios en total.

Configuración	Modo	Velocidad mínima	Velocidad máxima
C1. Lenta constante	Crucero	10	10
C2. Contraste	Crucero	10	40
C3. Rápida constante	Crucero	40	40
C4. Fluctuante	Variable	10	40

B.3. Fin de cada episodio

Cada episodio dura 60 s y se da por completado si en ese tiempo no se produce ninguno de estos fallos, que se comprueban en cada paso:

- **Colisión:** distancia entre centros menor que la distancia de colisión.
- **Fuera de la calzada:** el centro del seguidor sale de la calzada.
- **Guía perdido:** separación mayor que 1,0 m o negativa.

Una terminación del entorno propio por otra causa se registra como tal y no cuenta como completado.

B.4. Registro y métricas

En cada paso se registra el instante, la posición de ambos robots, la separación, si el seguidor está en el carril y la acción aplicada. De ahí salen cuatro métricas, y sólo cuatro, en conjunto y desglosadas por configuración:

Métrica	Definición
Completados	Número y porcentaje de episodios completados
Desenlaces	Número de episodios terminados por cada una de las causas de B.3
Separación	Porcentaje de pasos dentro de la banda objetivo, y separación media
Tiempo en el carril	Porcentaje de pasos en los que el seguidor está en el carril

